Sharing Databases in Microservices: Anti-Pattern or Practical?

Decomposition of Schema Coupling, Blast Radius Contention, Logical vs Physical Isolation, and Migration Patterns

EXECUTIVE SUMMARY

Sharing a database across microservices is the fastest shortcut to creating a 'Distributed Monolith'. While it simplifies joins and transactions initially, it introduces silent schema coupling, bypasses domain invariants, and creates a catastrophic shared failure domain. True microservice autonomy requires strict encapsulated persistence.

1. THE SPECTRUM OF DATABASE SHARING



2. WHY SHARED TABLES DESTROY MICROSERVICES (THE 3 CORE FAILURES)

1. Schema Lock-in Critical

Altering a column requires synchronizing code changes and release schedules across multiple teams. Independent CI/CD deployments are rendered impossible.

2. Domain Bypassing Severe

Direct SQL writes bypass business validation, state machines, and auditing logic implemented in the owning service's application code.

3. Contention Cascades High

A poorly indexed reporting query or table lock triggered by Service A starves connection pools and blocks critical OLTP transactions in Service B.

3. LOGICAL VS. PHYSICAL ISOLATION: TRADE-OFF COMPARISON

Dimension	Logical Isolation (Shared DB Cluster)	Physical Isolation (Database per Service)
Cost & Hardware	Low cost; pooled RAM and CPU resources.	Higher cost; multiple small instances.
Blast Radius	High; cluster OOM takes down all services.	Zero; failure is strictly fenced to one service.
Data Technology	Restricted to single DB engine (e.g. Postgres).	Polyglot (Postgres, Mongo, Neo4j, Cassandra).

4. PRAGMATIC EXCEPTIONS: WHEN SHARING IS ACCEPTABLE

While architectural purity favors physical separation, pragmatic constraints justify temporary or controlled sharing:

Acceptable Pragmatic Patterns

- **Static Reference Data:** Read-only lookup tables (countries, currencies) replicated or accessed with strict read-only DB users.
- **Strangler Fig Early Phase:** Deploying service code first while retaining DB access, before executing data migration.
- **Resource-Constrained Startups:** Running multiple isolated logical schemas on a single managed RDS cluster.

Unacceptable Toxic Patterns

- **Cross-Service Write Access:** Allowing multiple services to perform `INSERT` or `UPDATE` on identical tables.
- **Cross-Schema SQL Joins:** Writing queries that execute cross-database joins (e.g. `FROM orders JOIN users`).
- **Bypassing Domain Events:** Polling another service's database directly instead of subscribing to message queues.

5. THE MIGRATION PROTOCOL: DECOUPLING SHARED DATABASES

Decomposing a shared database in production without maintenance windows requires an incremental 5-phase migration:

Zero-Downtime Data Decoupling Sequence

1. **Step 1: API Encapsulation:** Prohibit direct database access; force all other services to call Service A's gRPC/REST endpoints.
2. **Step 2: Provision Target Datastore:** Deploy dedicated database instance configured for Service A's specific access patterns.
3. **Step 3: Dual-Write & Backfill:** Service A writes to both old and new databases simultaneously; a background batch job copies historical rows.
4. **Step 4: Change Data Capture (CDC):** Alternatively, stream real-time binlog events via Debezium/Kafka to replicate data with zero write overhead.
5. **Step 5: Cut-Over & Drop:** Point read traffic to new DB; deprecate writes to legacy DB; drop old tables.

6. DECLARATIVE DATABASE PERMISSIONS (POSTGRESQL SCHEMA FENCING)

```
-- Enforcing Logical Isolation in PostgreSQL
CREATE SCHEMA orders_schema;
CREATE ROLE orders_user LOGIN PASSWORD 'secure_pass';
GRANT USAGE, CREATE ON SCHEMA orders_schema TO orders_user;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA orders_schema TO orders_user;

-- Explicitly Revoke Cross-Schema Access:
REVOKE ALL PRIVILEGES ON SCHEMA inventory_schema FROM orders_user;
```

7. DISTRIBUTED CONSISTENCY WITHOUT SHARED ACID

Eliminating shared databases eliminates multi-table relational ACID transactions. Distributed workflows must adopt the **Saga Pattern**:

Orchestrated Saga (Centralized)

A central orchestrator (Temporal / Step Functions) executes discrete steps: OrderCreated → ReserveInventory → ProcessPayment. On failure, it explicitly calls compensating rollback endpoints.

Choreographed Saga (Event-Driven)

Services publish domain events to Kafka. Inventory listens to OrderCreated and reserves items; Payment listens to InventoryReserved and charges cards. Rollbacks require inverse events.

8. THE TRANSACTIONAL OUTBOX PATTERN

To atomically update local state and publish domain events without distributed 2-Phase Commit (2PC):

```
// Transactional Outbox in Go (ACID Guarantee)
tx, _ := db.BeginTx(ctx, nil)
defer tx.Rollback()

// 1. Mutate local domain entity
tx.Exec("INSERT INTO orders (id, customer_id, total) VALUES (?, ?, ?)", order.ID, order.CustomerID, order.Total)

// 2. Insert event into outbox table in the SAME atomic transaction
tx.Exec("INSERT INTO outbox (id, aggregate_type, event_type, payload) VALUES (?, ?, ?, ?)",
        uuid.New(), "Order", "OrderCreated", orderJSON)

tx.Commit() // Atomically commits both!
// 3. Debezium reads DB binlog and relays event to Kafka with At-Least-Once guarantee.
```

9. PRODUCTION ARCHITECTURAL VERIFICATION CHECKLIST

Database Decoupling Readiness Gates

1. **Zero Shared DB Users:** Every microservice connects with unique database credentials restricted to its own schema.
2. **Linter Rule on Cross-Joins:** Query analysis tools flag and block queries referencing tables outside the bounded context.
3. **Outbox Implementation:** Services writing state and producing events must utilize transactional outbox patterns.
4. **Saga Compensating Logic:** For every mutating distributed workflow step, automated rollback unit tests must pass in CI.